

Set de Pruebas

Jonathan Alexander Puentes Segura

1. Contest Scoreboard

Análisis General del Problema

El algoritmo procesa líneas con formato `equipo problema tiempo veredicto`.

- Solo los envíos con veredicto 'C' (Correcto) suman un problema resuelto.
- Los envíos con veredicto 'I' (Incorrecto) previos al primer 'C' de ese problema acumulan una penalización de **20 minutos** por cada incorrecto.
- Los veredictos 'R', 'U', 'E' se ignoran completamente.
- El ordenamiento final es:
 1. Mayor número de problemas resueltos.
 2. Menor penalización total (tiempo del 'C' + 20 * número de 'I' previos).
 3. Menor número de equipo.
- La entrada termina con un 0 en una línea.
- **Los equipos sin problemas resueltos NO deben aparecer en la salida**, aunque hayan realizado envíos incorrectos.

Riesgos principales:

- No acumular penalizaciones después del primer 'C'.
- No mostrar equipos con 0 problemas.
- Ordenar correctamente por los tres criterios.
- Terminar correctamente al leer 0.

Casos de Prueba

ID	Escenario	Entrada	Validación
CP-01	Un participante resuelve un problema directamente	1 1 10 C 0	Salida: 1 1 10. El equipo 1 tiene 1 problema y penalización 10.
CP-02	Incorrecto antes del correcto	1 1 10 I 1 1 20 C 0	Salida: 1 1 40 (20 + 20 de penalización).
CP-03	Varios incorrectos antes del correcto	1 1 10 I 1 1 15 I 1 1 20 I 1 1 30 C 0	Salida: 1 1 90 (30 + 3*20 = 90).
CP-04	Incorrecto después del correcto no penaliza	1 1 10 I 1 1 20 C 1 1 30 I 0	Salida: 1 1 40 (20 + 20 de penalización por el único incorrecto previo). El segundo incorrecto no suma.
CP-05	Problema no resuelto no genera penalización	1 1 10 I 1 2 20 I 0	No aparece ningún equipo (0 problemas resueltos).
CP-06	R, U y E no afectan el marcador	1 1 10 R 1 2 20 U 1 3 30 E 0	No aparece ningún equipo.
CP-07	Varias soluciones del mismo problema	1 1 10 C 1 1 20 C 0	Salida: 1 1 10. Solo se cuenta el primer correcto.
CP-08	Varios problemas resueltos	1 1 10 C 1 2 20 C 1 3 30 C 0	Salida: 1 3 60 (3 problemas, penalización 10+20+30=60).
CP-09	Orden por cantidad de problemas	1 1 10 C 2 1 10 C 2 2 20 C 0	Salida: 2 2 30 (2 problemas) y luego 1 1 10 (1 problema).

ID	Escenario	Entrada	Validación
CP-10	Desempate por penalización	1 1 50 C 2 1 30 C 0	Salida: 2 1 30 y luego 1 1 50 (menor penalización primero).
CP-11	Desempate por número de equipo	1 1 30 C 2 1 30 C 0	Salida: 1 1 30 y luego 2 1 30 (menor número de equipo primero).
CP-12	Participante con submissions pero sin soluciones	5 1 10 I 0	No aparece el equipo 5.
CP-13	Solo se muestran participantes con submissions	3 1 10 C 0	Salida: 3 1 10 (el equipo 3 sí aparece).
CP-14	Penalización acumulada de varios problemas	1 1 10 I 1 1 20 C 1 2 30 C 0	Salida: 1 2 70 (problema1: 20+20=40; problema2: 30; total 70).
CP-15	Rango de entradas completo	Múltiples equipos y problemas (caso extenso)	Se verifica que el orden y los cálculos coinciden con el estándar.
CP-16	Terminación con 0	1 1 10 C 0	El programa finaliza inmediatamente al leer 0, sin procesar líneas adicionales.
CP-17	Equipos sin resolver NO se muestran	1 1 10 I 2 1 20 I 0	No se imprime ninguna línea (ambos equipos tienen 0 problemas).
CP-18	Ordenación mixta (3 equipos)	2 1 10 C 1 1 5 C 1 2 5 C 3 1 1 C 0	Salida esperada: 3 1 1 1 2 10 2 1 10 (el equipo 1 tiene 2 problemas y penalización 10, el 2 tiene 1 y penalización 10).

2. $3n + 1$ / Collatz

Análisis General del Problema

Para cada par (i, j) , se debe calcular la longitud de la secuencia de Collatz para **todos** los números entre $\min(i, j)$ y $\max(i, j)$, y devolver la **longitud máxima** encontrada.

- La salida debe mantener el **orden original** de la entrada: se imprime i, j y el máximo, tal como se leyeron.
- La secuencia se define: si n es par, $n = n/2$; si es impar, $n = 3*n + 1$; la longitud incluye el número inicial y el 1 final.
- **Riesgos principales:**
 - Desbordamiento de enteros (32 bits) al calcular $3*n + 1$ (usar `long`).
 - Rendimiento en rangos grandes (recomendable memoización).
 - Manejo correcto de rangos invertidos ($i > j$).

Casos de Prueba

ID	Escenario	Entrada	Validación
CP-01	Número 1	1 1	Salida: 1 1 1 (longitud 1).
CP-02	Rango pequeño	1 10	Salida: 1 10 20 (máximo conocido para 1..10).
CP-03	Un solo número	22 22	Salida: 22 22 16 (longitud de 22).
CP-04	Rango normal	10 20	Salida: 10 20 21 (máximo entre 10 y 20).
CP-05	Rango invertido	20 10	Salida: 20 10 21 (mismo máximo, pero manteniendo orden 20 10).
CP-06	Dos números consecutivos	1 2	Salida: 1 2 2 (longitud de 1=1, de 2=2 $\rightarrow$ máximo 2).
CP-07	Número par	2 2	Salida: 2 2 2.
CP-08	Número impar	3 3	Salida: 3 3 8.
CP-09	Rango medio	100 200	Salida: 100 200 125 (máximo conocido).
CP-10	Rango medio 201-210	201 210	Salida: 201 210 89 (máximo en ese rango).
CP-11	Rango grande	900 1000	Salida: 900 1000 174 (máximo conocido).

ID	Escenario	Entrada	Validación
CP-12	Rango invertido grande	1000 900	Salida: 1000 900 174 (orden invertido).
CP-13	Varias consultas	1 10 100 200 201 210 900 1000	Salida: cuatro líneas con los máximos respectivos, en el mismo orden de entrada.
CP-14	Rango extremo (rendimiento)	1 100000	La ejecución debe finalizar en menos de 1 segundo (en Java/C++). El máximo esperado es 351.
CP-15	Número con alto crecimiento	99999 99999	No debe haber desbordamiento. Longitud esperada $\sim$ 225-230 ciclos.
CP-16	Múltiples consultas desordenadas	10 1 20 10	Salida: 10 1 20 20 10 21 (cada línea con el par original).

3. DNA / Culture Growth

Análisis General del Problema

Se simulan **50 días** en **40 platos** (posiciones 0 a 39).

- Para cada día y cada plato, se calcula la **suma de las poblaciones** del plato mismo, el de la izquierda y el de la derecha.
- Los vecinos fuera de los bordes (posición -1 y 40) se consideran siempre **0**.
- La suma (valor entre 0 y 9) se usa como índice en el arreglo **DNA** de 10 elementos para obtener la nueva población.
- La simulación genera el estado final después de 50 días (o se imprime día a día según la versión).
- **Riesgos principales:**
 - Manejo incorrecto de bordes (acceder a índices negativos o fuera de rango).
 - Número de días exacto (50) y número de platos (40) fijos.
 - Acumulación de errores en la suma si no se usa una copia del estado anterior.

Casos de Prueba

ID	Escenario	Entrada	Validación
CP-01	DNA que elimina toda la población	DNA: 0 0 0 0 0 0 0 0 0 0 Inicial: cualquier	A los 50 días, todas las posiciones deben ser 0.
CP-02	DNA que produce máxima población	DNA: 3 3 3 3 3 3 3 3 3 3 Inicial: cualquier	Todas las posiciones se vuelven 3 (si la suma es 9 $\rightarrow$ DNA[9]=3).
CP-03	Caso oficial	DNA: 0 1 2 0 1 3 3 2 3 0 Inicial: 0 ... 0 (40 posiciones)	Se debe verificar con la salida oficial del problema (conocida).
CP-04	Solo DNA[0]	DNA: 1 0 0 0 0 0 0 0 0 0 Inicial: cualquier	Todas las poblaciones se vuelven 0, porque cualquier suma ¿0 apunta a DNA[0]=1, pero si la suma es 0 $\rightarrow$ DNA[0]=1, entonces se mantiene 1 en posiciones con vecinos 0. Validar evolución.

ID	Escenario	Entrada	Validación
CP-05	Solo DNA[1]	DNA: 0 1 0 0 0 0 0 0 0 0	Similar, verificar que solo sobreviven donde la suma sea exactamente 1.
CP-06	DNA con valores directos	DNA: 0 1 2 3 0 0 0 0 0 0	Verificar que la población se transforma según la suma.
CP-07	Todos los valores 1	DNA: 1 1 1 1 1 1 1 1 1 1	Todas las posiciones se mantienen en 1 (siempre que la suma sea 0..9, DNA[suma]=1).
CP-08	Propagación desde el centro	DNA: 0 3 0 0 0 0 0 0 0 0 Inicial: 0 ... 3 en posición 20 ... 0	Verificar que el 3 se propaga a los vecinos según regla.
CP-09	Valores variados	DNA: 2 1 3 0 2 1 0 3 2 1	Comprobar la evolución con una entrada variada.
CP-10	Límite izquierdo	Cualquier DNA, estado inicial con borde izquierdo !=0	Verificar que el vecino izquierdo del plato 0 se considera 0.
CP-11	Límite derecho	Cualquier DNA, estado inicial con borde derecho !=0	Verificar que el vecino derecho del plato 39 se considera 0.
CP-12	Estado inicial	Cualquier DNA, estado inicial aleatorio	Verificar que la simulación parte del estado dado.
CP-13	50 días	Cualquier DNA	Se generan exactamente 50 días de evolución.
CP-14	Suma máxima en vecinos	DNA: 0 0 0 0 0 0 0 0 0 1 Inicial: 3 3 3 ... (40 veces)	Día 1: cada plato suma 9 (3+3+3), usa DNA[9]=1, toda la población se convierte en 1.
CP-15	Estado inicial ya muerto	DNA cualquiera Inicial: 0 0 ... 0	Todos los días se mantiene 0.
CP-16	Borde izquierdo y derecho	DNA: 1 0 0 0 0 0 0 0 0 0 Inicial: 0 1 0 0 ...	Plato 0: suma = 0 (propio) + 1 (derecha) + 0 (borde) = 1 → DNA[1]=0, se extingue; verificar que no se accede a índice -1.

4. The Snail

Análisis General del Problema

El caracol intenta subir una pared de altura H . Cada día:

- La **subida del día** es $U - (\text{fatiga_acumulada})$, donde la fatiga comienza en 0 y se incrementa diariamente en $F\%$ de U (es decir, $\text{fatiga} += U * F / 100$). La subida no puede ser negativa (si la fatiga supera U , la subida es 0).
- **Primero** se verifica si la altura alcanzada ($\text{altura_actual} + \text{subida_día}$) es mayor o igual a H . Si es así, el caracol **éxito** y termina.
- Si no, se resta el deslizamiento nocturno D ; si la altura resultante es menor que 0, se considera **fracaso**.
- La simulación continúa hasta éxito o fracaso.
- **Riesgos principales:**
 - Orden de operaciones: comprobar éxito **antes** del deslizamiento.
 - La fatiga se aplica cada día, incluso si la subida se vuelve 0.
 - Bucles infinitos si $U \leq D$ y nunca se alcanza la altura (debe detectarse fracaso cuando la altura disminuye o se estanca).

Casos de Prueba

ID	Escenario	Entrada	Validación
CP-01	Caso oficial	6 3 1 10	Éxito en el día 3 (simulación estándar).
CP-02	Fracaso por fatiga	2 1 50 50	Fracaso en el día 1 (subida 0, desliza y queda negativo).
CP-03	Fatiga alta	5 3 14 50	Fracaso en el día 2 o 3 (según simulación).
CP-04	Muchos días	6 4 1 50	Éxito en el día 7 (conocido).
CP-05	Fatiga baja	6 3 1 1	Éxito en el día 3 (fatiga insignificante).
CP-06	Caso pequeño	1 1 1 1	Éxito en el día 1.
CP-07	Éxito el primer día	10 15 1 10	Éxito en el día 1.
CP-08	Supera exactamente la altura	5 5 1 10	Éxito en el día 1 (sube 5 y alcanza $H=5$).
CP-09	La subida llega a cero	10 2 1 100	Fracaso porque la subida se vuelve 0 rápidamente.

ID	Escenario	Entrada	Validación
CP-10	U mayor que H	5 10 3 50	Éxito en el día 1.
CP-11	U igual a D	5 3 3 10	Fracaso porque después del primer día no avanza (sube 3, baja 3).
CP-12	D mayor que U	10 2 3 10	Fracaso en el día 1 (sube 2, baja 3, queda negativo).
CP-13	Múltiples casos en líneas	Varias líneas consecutivas	Cada línea se procesa independientemente y se imprime el resultado correcto.
CP-14	Subida negativa por fatiga	10 2 100 1	Día 1: subida = 0 (por fatiga), no alcanza, desliza 1 → altura -1, fracaso día 1.
CP-15	Éxito justo al tocar el borde	10 10 0 1	Día 1: sube 10, alcanza H, éxito sin deslizarse.
CP-16	Ciclo de fatiga que se estabiliza	100 15 20 10	Fracaso después de varios días (cuando subida diaria ¡deslizamiento). Verificar que no hay bucle infinito.

5. Integer Steps

Análisis General del Problema

Dada la distancia $d = |x - y|$, se debe encontrar el número mínimo de pasos n tal que la suma $1 + 2 + \dots + n$ sea mayor o igual a d y que la diferencia (suma - d) sea par.

- La secuencia de pasos es 1, 2, 3, ..., n .
- La solución se obtiene resolviendo $n*(n+1)/2 \geq d$ y ajustando n si la paridad no coincide (incrementando de 1 en 1 o en 2).
- **Riesgos principales:**
 - Desbordamiento en el cálculo de $n*(n+1)/2$ para distancias cercanas a $2^{31} \cdot Usar long$.
 - Manejo de distancias 0 y 1 correctamente.
 - Lógica de paridad: si la diferencia es impar, se necesita un paso adicional o dos, dependiendo de la paridad de n .

Casos de Prueba

ID	Escenario	Entrada	Validación
CP-01	Misma posición	0 0	Pasos = 0.
CP-02	Distancia 1	0 1	Pasos = 1 (1 $\geq$ 1, diff 0 par).
CP-03	Distancia 2	0 2	Pasos = 3. (Suma 1+2=3, diff=1 impar $\rightarrow$ n=3 da suma 6, diff=4 par).
CP-04	Distancia 3	0 3	Pasos = 2. (n=2 suma 3, diff 0 par).
CP-05	Distancia 4	0 4	Pasos = 3 (suma 6, diff 2 par).
CP-06	Distancia 5	0 5	Pasos = 4. (n=3 suma 6, diff=1 impar $\rightarrow$ n=4 suma 10, diff=5 par).
CP-07	Distancia 6	0 6	Pasos = 3 (suma 6, diff 0 par).
CP-08	Caso oficial	45 48	Distancia 3 $\rightarrow$ pasos 2.
CP-09	Caso oficial	45 49	Distancia 4 $\rightarrow$ pasos 3.
CP-10	Caso oficial	45 50	Distancia 5 $\rightarrow$ pasos 4.
CP-11	Misma posición con otro valor	100 100	Pasos 0.

ID	Escenario	Entrada	Validación
CP-12	Distancia 1 con otros valores	100 101	Pasos 1.
CP-13	Distancia 2	100 102	Pasos 3 (como en CP-03).
CP-14	Distancia 3	100 103	Pasos 2.
CP-15	Distancia 4	100 104	Pasos 3.
CP-16	Distancia 5	100 105	Pasos 4.
CP-17	Valor máximo del rango	2147483646 2147483647	Distancia 1 $\rightarrow$ pasos 1.
CP-18	Rango grande	0 2147483646	Distancia 2.147.483.646. El cálculo debe usar <code>long</code> y no desbordar. El resultado esperado es 65536 (se conoce que para $d=2^{31}-2$, $n \approx 65536$).
CP-19	Distancia muy grande	0 1000000000	Pasos esperados = 44721 (raíz de $2*1e9$). Verificar que no hay desbordamiento.
CP-20	Justo después de un cuadrado perfecto	0 7	Distancia 7: $n=3$ suma 6 (¡7), $n=4$ suma 10, $\text{diff}=3$ impar $\rightarrow$ $n=5$ suma 15, $\text{diff}=8$ par $\rightarrow$ pasos 5.
CP-21	Distancia exacta triangular	0 6	Distancia 6: $n=3$ suma 6, diff 0 par $\rightarrow$ pasos 3.

Criterios Generales de Aceptación

- Los casos normales, límites y condiciones de error o terminación producen el resultado esperado.
- Los problemas con rangos se validan tanto en $x \leq y$ como en $x > y$ cuando el enunciado lo permite.
- Los algoritmos iterativos incluyen casos de terminación rápida y casos de muchas iteraciones.
- La cobertura de código JaCoCo debe contemplar:
 - Ramas `if/else` por ejemplo, veredictos, paridad, condiciones de éxito/fracaso.
 - Condiciones de borde acceso a arreglos, límites de enteros.
 - Acumulación de resultados penalizaciones, máximos, poblaciones.
- Cuando sea posible, se separan pruebas unitarias de funciones auxiliares y pruebas de integración de entrada/salida.